

北京林业大学

数据库原理与应用

本章目录 CONTENTS

- | 数据库并发性的含义
- | 事务及其特性
- | 并发操作带来的不一致性
- | 封锁协议
- | 总结

数据库并发性的含义



数据库并发性的含义



- ◆ 多个用户同时访问数据库，如果对并发操作不加控制可能会产生不正确的数据，破坏数据的完整性。
- ◆ 并发控制就是解决这类问题，以保持数据库中数据的一致性，即在任何一个时刻数据库都将以相同的形式给用户提供服务。

事务及其特性



事务 (Transaction)

- ◆ 事务是数据库系统中执行的一个工作单位，它是由用户定义的一组操作序列。
- ◆ 一个事务可以是一组SQL语句、一条SQL语句或整个程序，一个应用程序可以包括多个事务。

定义事务的语句有三条：

BEGIN TRANSACTION

事务的开始

COMMIT

事务的提交

ROLLBACK

事务的回滚



事务 (Transaction)

原子性 (Atomicity)

一个事务是一个不可分割的工作单位，事务在执行时，应该遵守“要么不做，要么全做” (Nothing or All) 的原则，即不允许完成部分的事务。

一致性 (Consistency)

事务对数据库的作用是数据库从一个一致状态转变到另一个一致状态。所谓数据库的一致状态是指数据库中的数据满足完整性约束。

隔离性 (Isolation)

如果多个事务并发地执行，应像各个事务独立执行一样，一个事务的执行不能被其他事务干扰。

持久性 (Durability)

持久性指一个事务一旦提交，它对数据库中数据的改变就应该是持久的，即使数据库因故障而受到破坏，DBMS也应该能够恢复。

并发操作带来的不一致性



举例说明不一致问题的产生过程

例：

并发取款操作。假设存款余额 $R=1000$ 元，甲事务 T_1 取走存款100元，乙事务 T_2 取走存款200元，如果正常操作，即甲事务 T_1 执行完毕再执行乙事务 T_2 ，存款余额更新后应该是700元。但是如果按照如下顺序操作，则会有不同的结果：



举例说明不一致问题的产生过程

- 01 甲事务T1读取存款余额 $R=1000$ 元;
- 02 乙事务T2读取存款余额 $R=1000$ 元;
- 03 甲事务T1取走存款100元, 修改存款余额 $R=R-100=900$, 把 $R=900$ 写回到数据库;
- 04 乙事务T2取走存款200元, 修改存款余额 $R=R-200=800$, 把 $R=800$ 写回到数据库;

结果两个事务共取走存款300元, 而数据库中的存款却只少了200元。得到这种错误的结果是由甲乙两个事务并发操作引起的。



引起数据不一致性的主要问题

数据库的并发操作导致的数据库不一致性主要有以下三种：

丢失更新 (Lost Update)

污读 (Dirty Read)

不可重读 (Unrepeatable Read)



引起数据不一致性的主要问题

数据库的并发操作导致的数据库不一致性主要有以下三种：

丢失更新 (Lost Update)

时 间	事务 T_1	数据库中R的值	事务 T_2
t_0		1000	
t_1	FIND R		
t_2			FIND R
t_3	$R=R-100$		
t_4			$R=R-200$
t_5	UPDATE R		
t_6		900	UPDATE R
t_7		800	

当两个事务 T_1 和 T_2 读入同一数据，并发执行修改操作时， T_2 把 T_1 或 T_1 把 T_2 的修改结果覆盖掉，造成了数据的丢失更新问题，导致数据的不一致。



引起数据不一致性的主要问题

数据库的并发操作导致的数据库不一致性主要有以下三种：

污读 (Dirty Read)

时 间	事务 T_1	数据库中R的值	事务 T_2
t_0		1000	
t_1	FIND R		
t_2	$R=R-100$		
t_3	UPDATE R		
t_4		900	FIND R
t_5	ROLLBACK		
t_6		1000	

事务 T_1 更新了数据R，事务 T_2 读取了更新后的数据R，事务 T_1 由于某种原因被撤销，修改无效，数据R恢复原值。事务 T_2 得到的数据与数据库的内容不一致，这种情况称为“污读”。



引起数据不一致性的主要问题

数据库的并发操作导致的数据库不一致性主要有以下三种：

不可重读 (Unrepeatable Read)

时 间	事务 T_1	数据库中R的值	事务 T_2
t_0		1000	
t_1	FIND R		
t_2			FIND R
t_3			$R=R-200$
t_4			UPDATE R
t_5		800	

事务 T_1 读取了数据R，事务 T_2 读取并更新了数据R，当事务 T_1 再读取数据R以进行核对时，得到的两次读取值不一致，这种情况称为“不可重读”。

封锁协议



封锁分类

实现并发控制的方法主要有两种：

封锁 (Lock) 技术和时标 (Timestamping) 技术，重点介绍封锁技术。

封锁的定义

所谓封锁就是当一个事务在对某个数据对象（可以是数据项、记录、数据集以至整个数据库）进行操作之前，必须获得相应的锁，以保证数据操作的正确性和一致性。



封锁分类

基本的封锁类型有两种

排它型封锁 (Exclusive Lock)

排它型封锁又称写封锁，简称为X封锁，它采用的原理是禁止并发操作。

共享封锁 (Share Lock)

共享封锁又称读封锁，简称为S锁，它采用的原理是允许其他用户对同一数据对象进行查询，但不能对该数据对象进行修改。



使用封锁协议的过程

封锁协议的定义 (Lock Protocol)

- 封锁可以保证合理地进行并发控制，保证数据的一致性。
- 在封锁时，要考虑一定的封锁规则，例如，何时开始封锁、封锁多长时间、何时释放等，这些封锁规则称为封锁协议。



使用封锁协议的过程

通过三级封锁协议在不同程度上给予解决：

一级封锁协议

二级封锁协议

三级封锁协议



使用封锁协议的过程

一级封锁协议

事务T在修改数据对象之前必须对其加X锁，直到事务结束。

时 间	事务T ₁	数据库中R的值	事务T ₂
t ₀	XLOCK R	1000	
t ₁	FIND R		
t ₂			XLOCK R
t ₃	R=R-100		WAIT
t ₄	UPDATE R		WAIT
t ₅	UNLOCK X	900	WAIT
t ₆			XLOCK R
t ₇			R=R-200
t ₈			UPDATE R
t ₉		700	UNLOCK X



使用封锁协议的过程

二级封锁协议

在一级封锁协议的基础上，另外加上事务T在读取数据R之前必须先对其加S锁，读完后释放S锁。

时 间	事务T ₁	数据库中R的值	事务T ₂
t ₀	XLOCK R	1000	
t ₁	FIND R		
t ₂	R=R-100		
t ₃	UPDATE R		
t ₄		900	SLOCK R
t ₅	ROLLBACK		WAIT
t ₆	UNLOCK X	1000	SLOCK R
t ₇			FIND R
t ₈			UNLOCK S



使用封锁协议的过程

三级封锁协议

在一级封锁协议的基础上，另外加上事务T在读取数据R之前必须先对其加S锁，读完后并不释放S锁，而直到事务T结束才释放。

时 间	事务T ₁	数据库中R的值	事务T ₂
t ₀		1000	
t ₁	SLOCK R		
t ₂	FIND R		
t ₃			XLOCK R
t ₄	COMMIT		WAIT
t ₅	UNLOCK S		WAIT
t ₆			XLOCK R
t ₇			FIND R
t ₈			R=R-200
t ₉			UPDATE R
t ₁₀		800	UNLOCK X



封锁粒度

封锁粒度的含义

封锁粒度指封锁的单位。

根据对数据的不同处理，封锁的对象可以是这样一些逻辑单元：字段、记录、表、数据库等，封锁的数据对象的大小叫封锁粒度。



封锁粒度

封锁粒度越小，系统中能够被封锁的对象就越多，并发度越高，但封锁机构复杂，系统开销也就越大。

VS

封锁粒度越大，系统中能够被封锁的对象就越少，并发度越低，封锁机构越简单，相应系统开销也就越小。



封锁所产生的额问题

死锁和活锁

活锁 (Livelock)

当某个事务请求对某一数据进行排它性封锁时，由于其他事务对该数据的操作而使这个事务处于永久等待状态，这种状态称为活锁。

时 间	事务T ₁	事务T ₂	事务T ₃	事务T ₄
t ₀	LOCK R			
t ₁		LOCK R		
t ₂		WAIT	LOCK R	
t ₃	UNLOCK	WAIT	WAIT	LOCK R
t ₄		WAIT	LOCK R	WAIT
t ₅		WAIT		WAIT
t ₆		WAIT	UNLOCK	WAIT
t ₇		WAIT		LOCK R
t ₈		WAIT		



封锁所产生的额问题

死锁和活锁

死锁 (Deadlock)

在同时处于等待状态的两个或多个事务中，其中的每一个在它能够进行之前，都等待着某个数据，而这个数据已被它们中的某个事务所封锁，这种状态称为死锁。

时 间	事务 T_1	事务 T_2
t_0	LOCK R_1	
t_1		LOCK R_2
t_2		
t_3	LOCK R_1	
t_4	WAIT	
t_5	WAIT	LOCK R_1
t_6	WAIT	WAIT
t_7	WAIT	WAIT



解决封锁的方法

死锁的预防

一次加锁法:

一次性对所有资源加锁，如果有一个资源无法加锁，则等待。

顺序加锁法:

规定资源的序号，按序号加锁



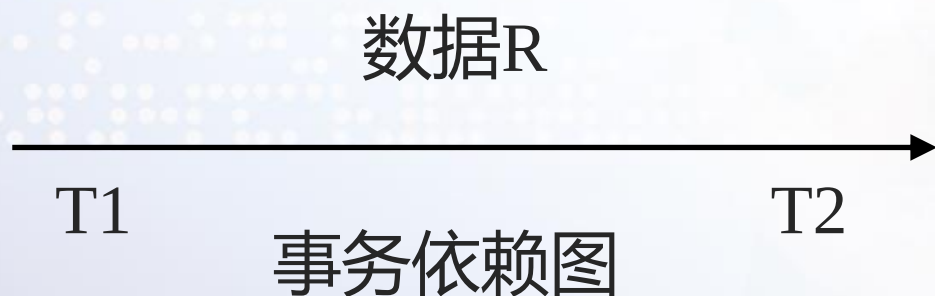
解决封锁的方法

死锁的预防

死锁的诊断与解除

如果在事务依赖图中沿着箭头方向存在一个循环，那么死锁的条件就形成了，系统就会出现死锁。

选择一个处理死锁代价最小的事务，将其撤销以解除死锁。



总结



知识点总结

